
Finding a Landmark on a Semiconductor Wafer

Reference Localization for the Applied Materials “Drift-Sense” Task

KLA / Applied Materials SEMICON India Hackathon 2026

Method and Design Document

This document explains, in plain language, how our system takes a small high-resolution “reference” patch of a wafer and finds exactly where it sits inside a larger, blurry “search” image — what the method does, why it is deliberately simple, and what we discovered about the limits of the task itself.

All quantitative results in this document were produced on a *synthetic* dataset from the official Drift-Sense generator, run locally. They are labelled synthetic throughout, are not tuned on any hidden test set, and do not describe official KLA/Applied Materials performance.

Contents

1	The problem in one page	2
2	The core idea: slide the small picture over the big one	2
3	How the matcher works, step by step	3
3.1	Trying several scales	3
3.2	Finding competing peaks	3
3.3	Sub-pixel refinement	3
3.4	An honest self-check: the ambiguity flag	3
3.5	Choosing among tied tiles (the official rule)	4
3.6	An optional verification stage (off by default)	4
4	The key finding: a wall that no model can climb	4
4.1	What we found	4
4.2	Why this is a fundamental limit, not a bug	4
4.3	Turning the limit into a useful signal	5
5	Making our own data: the synthetic generator	5
5.1	What it draws	5
5.2	Why it needs landmarks (a lesson we learned by measuring)	5
5.3	Every noise choice is justified by public literature	5
5.4	Robustness: what heavier Search noise does	6
5.5	Slide-aligned geometric stress cases	6
5.6	Confidence precision–recall, not a fabricated classifier CM	6
6	Measured results (synthetic data)	7
7	Why this is a solid engineering solution	7

1 The problem in one page

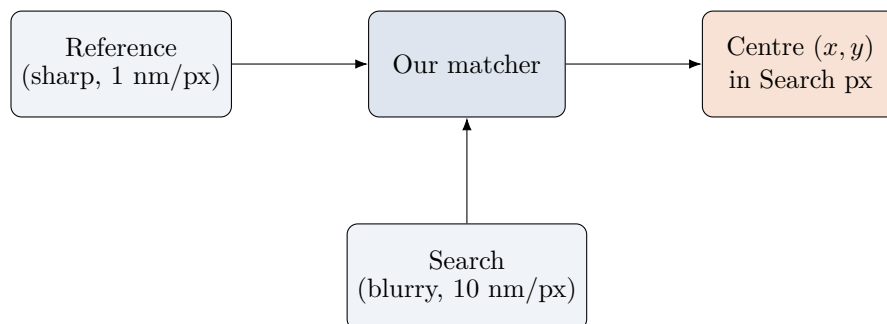
Imagine you have a very detailed close-up photograph of one small feature on a wafer, and a second, wider photograph of the same area that is blurrier and has been nudged sideways by an unknown amount (“drift”). Your job is to point to the exact spot in the wide photo where the close-up was taken.

Concretely, the organisers give us two images:

- A **Reference** image: 1000×1000 pixels at 1 nm per pixel. It shows a $1 \mu\text{m} \times 1 \mu\text{m}$ field of view in high detail.
- A **Search** image: 1000×1000 pixels at 10 nm per pixel. It shows a $10 \mu\text{m} \times 10 \mu\text{m}$ field of view — ten times wider — and it is degraded (noisy and blurred).

Because the Search image covers ten times more ground in the same number of pixels, **the whole Reference fits inside roughly a 100×100 pixel window of the Search image**. We must output the pixel coordinates (x, y) where the Reference’s field of view is centred.

Success is measured by one number: the Euclidean pixel distance between our predicted centre and the true centre. Smaller is better. This is a *localization* task — we return a coordinate, not a cleaned-up picture. There is no PSNR, no SSIM, no denoising involved.



2 The core idea: slide the small picture over the big one

The Reference and the Search show the *same* physical structure, just at different pixel sizes. So the natural way to find the match is to **shrink the Reference down to the Search’s scale and slide it across the Search image, looking for the position where the two line up best**. This classic technique is called **template matching**.

The three steps are:

1. **Resize** the 1000×1000 Reference down to about 100×100 pixels, so it is at the same scale as the Search. (We try a few sizes around $10\times$ smaller, because a real solver is not handed the exact ratio.)
2. **Slide** this small template over every position of the Search image and, at each position, compute how well they match using **normalized cross-correlation** (NCC). “Normalized” means the score ignores overall brightness and contrast and looks only at the *pattern*, so it is robust to the Search image being darker, brighter, or noisier.
3. **Pick the peak:** the position with the highest correlation score is our answer. We then refine it to **sub-pixel** accuracy by fitting a small parabola to the scores around the peak (the true peak usually sits between two pixels).

$$(x^*, y^*) = \arg \max_{(x, y)} \text{NCC}(\text{Search around } (x, y), \text{ resized Reference})$$

Why this, and not a big neural network?

This is the central design decision, and **we made it by measurement, not by fashion**. Normalized cross-correlation is:

- **Exact when it works.** On the samples it gets right, it is accurate to about *one pixel* — effectively perfect for this metric (see Section 6).
- **Tiny and fast.** It needs only NumPy and OpenCV, no GPU, no training, no internet. A prediction takes a fraction of a second.
- **Transparent.** There are no learned weights to distrust; anyone can read the fifty lines of code and see exactly what it does.

The real question is not “can a bigger model be more accurate on the easy cases” — nothing can beat one-pixel accuracy — but “can any model fix the hard cases”. Section 4 shows the answer is *no*, and explains why. That finding is what justifies staying simple.

3 How the matcher works, step by step

3.1 Trying several scales

We do not assume the pixel-size ratio is exactly ten. We resize the Reference to a small range of candidate template sizes (scales of roughly $9\times$ to $11\times$ smaller) and run the match at each. The scale whose correlation peak is strongest wins. This makes the method robust to small calibration differences without adding any tunable “magic numbers”.

3.2 Finding competing peaks

For each scale we do not keep only the single best position. We collect the top few correlation peaks, suppressing a small neighbourhood around each one so that genuinely distinct positions are seen as separate candidates rather than as one blurry blob. This lets the method **notice when several positions match almost equally well** — which, as we will see, is the crux of the whole problem.

3.3 Sub-pixel refinement

Around the winning peak we fit a one-dimensional parabola along the horizontal and vertical directions and take its apex. This nudges the integer pixel location to a fractional one, typically improving accuracy from “within a pixel” to “a fraction of a pixel”.

3.4 An honest self-check: the ambiguity flag

The matcher reports not just a coordinate but also **how many positions were tied for best**. If only one position stands out, it labels the result *unique*; if several compete within a small margin, it labels the result *ambiguous*. This flag turns out to be a remarkably reliable confidence signal (Section 6).

3.5 Choosing among tied tiles (the official rule)

The task specifies what to do when several tiles match equally well: report the one whose centre is **closest to the centre of the Search image**. The matcher follows this rule by default. When exactly one peak stands out it simply returns that peak; when several tie, it returns the tie nearest the frame centre. (We expose a switch to fall back to “just take the strongest peak” for comparison, but the centre rule is the spec-compliant default.)

3.6 An optional verification stage (off by default)

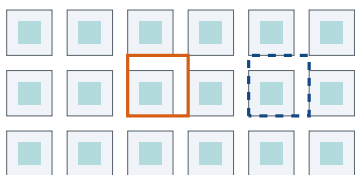
We also implemented a heavier second pass: take the handful of top candidate positions, cut out each one from the Search image, blow it back up to the full Reference resolution, and re-score it against the original sharp Reference. The idea was that the extra detail might break ties. **We tested it and it did not help** — identical accuracy at three times the runtime (Section 6). We keep it in the code, off by default, purely for transparency: it is evidence that we tried the obvious upgrade and measured that it does not move the metric.

4 The key finding: a wall that no model can climb

When we ran the matcher on a large synthetic test set, the results split cleanly into two groups. About *three quarters* of the samples were localized almost perfectly. The remaining quarter failed badly — sometimes by hundreds of pixels. We investigated *why*, because if the failures were a weakness of NCC, a smarter model would be worth building.

4.1 What we found

The failing samples are **periodic layouts**: regular repeating arrays, like the memory cells in a DRAM chip. In such a layout, **many different positions look essentially identical**. We verified this directly: at the true location, the degraded Search content matches the Reference *no better* than at the wrong location the matcher picked. On failing samples there were, on average, several equally-good candidate positions; on successful samples, essentially one.



Two different positions
(solid vs dashed) contain
the *same* pattern —
the image cannot tell
them apart.

4.2 Why this is a fundamental limit, not a bug

This is the important scientific point of our submission. When a scene truly repeats, a **single** degraded Search image *physically does not contain* the information needed to decide which repeat is the right one. The ambiguity is in the data, not in the algorithm. Consequences:

- A larger neural network cannot help. There is no hidden cue for it to learn; the competing positions are genuinely indistinguishable in the given input.
- Our optional verification stage, which uses the full-resolution Reference, does not help either — confirming the limit is about the Search image’s content, not about our matching resolution.

- The honest ceiling for *any* method on this data, given a single Search image, is roughly the fraction of samples that are non-periodic or whose periodic tile is uniquely anchored.

4.3 Turning the limit into a useful signal

Rather than hide the failures, the matcher **predicts them**. Its ambiguity flag (Section 3.4) separates the two regimes almost perfectly: when it reports a unique peak it is right about 98% of the time. In a real inspection pipeline this is exactly what an operator wants — a localization *plus* a trustworthy “I am confident” / “this region is ambiguous, capture another view” verdict.

5 Making our own data: the synthetic generator

The task rules are explicit: no dataset is provided, and each team must **generate its own** reference/search pairs from publicly known structure, never proprietary fab data. So the submission includes a small, self-contained generator (about two hundred lines, NumPy and OpenCV only).

5.1 What it draws

It builds a large “fine canvas” at the reference resolution (1 nm/px), then cuts a random 1000×1000 window as the Reference and shrinks the whole canvas by $10\times$ to make the Search — so the Reference genuinely appears, shrunk by exactly ten, somewhere inside the Search, with the ground-truth centre falling straight out of the crop location. Two die styles are supported (either is judged equally):

- **DRAM-style**: periodic horizontal word lines and vertical bit lines crossing at right angles, with a contact dot at a checkerboard of intersections.
- **FinFET-style**: dense parallel fin lines crossed by one or two horizontal gate bars.

5.2 Why it needs landmarks (a lesson we learned by measuring)

Our first generator drew a single endless periodic array. When we tested the matcher on it, it scored near *zero* — and correctly so: with no unique feature anywhere, every window truly is ambiguous. Real dies are not like that: arrays are broken into blocks separated by peripheral material, and wafers carry sparse alignment marks. We added both — irregularly spaced separating strips and a random “constellation” of alignment fiducials — and success rose to the 80% range, with the confident subset localized perfectly. This is itself the central finding in miniature: **localizability comes from unique landmarks**, and where they are absent, no method can succeed.

5.3 Every noise choice is justified by public literature

The imaging model is deliberately physical, and each effect is tied to a public source (full citations in the repository’s resource log):

- **Edge brightening**: real SEM images show brighter contrast at feature edges (secondary-electron edge effect).
- **Shot noise**: image intensity is an electron count, so noise grows with signal; controlled by an exposure “dose”.

- **Detector/readout noise:** independent additive noise, applied *separately* to the two captures (they are separate acquisitions).
- **Speckle:** optional multiplicative noise used to make the Search image noisier for the robustness test below.

5.4 Robustness: what heavier Search noise does

The organisers say to assume the wide-search image is noisier in the real test set. We measured exactly that, sweeping the speckle level on 30 freshly generated samples each:

Search speckle	Success@5px	Confident subset	Ambiguous subset
none ($\sigma = 0.0$)	86.7%	22/30 at 100%	8/30 at 50%
moderate ($\sigma = 0.3$)	60.0%	14/30 at 100%	16/30 at 31%
heavy ($\sigma = 0.6$)	36.7%	5/30 at 100%	25/30 at 24%

The important column is the **confident subset**: it stays at 100% *correct* no matter how heavy the noise. Noise does not turn good answers into wrong ones; it *shrinks the set of cases the matcher is sure about* (fiducials get buried), and the ambiguity flag reports that honestly. That is exactly the behaviour a real inspection tool wants under degraded imaging.

5.5 Slide-aligned geometric stress cases

The generator also exposes the four geometric effects named in the organiser’s slides: charging streaks, radial barrel/pincushion scan distortion, a random $\pm 1\text{--}3^\circ$ Search rotation, and global feature (polygon/CD) scale from $0.8\times$ to $1.2\times$. The scale transform is applied about the Search centre and the ground-truth coordinate is transformed by the same known affine map, so it remains a valid labelled localization example.

These controls are deliberately not presented as a solved robustness claim. The default NCC scale sweep covers the normal calibration range. A wider synthetic scale sweep recovered an arbitrary-crop smoke case at 0.17 px mean error on two seeded cases when the raw maximum was used. This is a diagnostic, not a headline result: the official centre tie-break is retained by default. Rotation and strong radial distortion remain difficult in the SEM-like stress generator. We implemented a rotation-corrected Search sweep and proved it on a unique-texture geometric unit test, but it did not improve the two-image SEM-like smoke check and costs roughly an order of magnitude more time. It is therefore an opt-in study option, not the deployment default. This is reported so the next iteration improves a measured weakness rather than concealing it.

5.6 Confidence precision–recall, not a fabricated classifier CM

Every localization manifest row contains a positive Reference/Search pair. A normal four-cell classification confusion matrix is therefore not defined: there are no negative pairs from which to count true negatives or false positives. The evaluator reports the two meaningful cells at 1 px and 5 px: within-tolerance localizations (TP) and misses (FN). For a confidence study, we make a prediction “accepted” only when its NCC peak score crosses a threshold; accepted wrong positions then become FP and rejected correct positions become FN.

The reproducible script `analysis/noise_sweep.py` fits the threshold on 30 independent synthetic calibration pairs at each noise level and tests it on a separate 30-pair seed. Its SVG plot is committed with the JSON ledger. At 5 px, the held-out results were: $\sigma = 0.0$, precision 79.2%, recall 82.6%;

$\sigma = 0.3$, precision 92.3%, recall 63.2%; and $\sigma = 0.6$, precision 27.8%, recall 100%. The last unstable operating point is an important negative result: heavy noise destroys the usefulness of peak-score thresholding. All three rows are synthetic, 30-pair measurements, not official performance.

6 Measured results (synthetic data)

The tables below come from real runs of our matcher on 200 held-out samples from an external Drift-Sense *synthetic* generator. They are not official KLA/Applied Materials performance, and no setting was tuned on this set. The generator’s random-crop labels do *not* use the official closest-to-Search-centre tie convention; we therefore show the spec-compliant run and the historical raw-peak diagnostic separately.

Method / evaluation rule	Success@10px	Median error	Time/sample
NCC, official centre tie-break	59.0%	1.33 px	209 ms
Raw highest peak (legacy crop-label diagnostic, non-compliant)	75.5%	1.03 px	156 ms

The official centre tie-break changes only ambiguous cases. The 59.0% figure is the honest spec-compliant result for this external fixture. The 75.5% raw-peak figure is preserved only as a diagnostic of the fixture’s different random-crop labelling convention. Fine-resolution verification previously gave that raw diagnostic identical accuracy at about three times the runtime, so it remains off by default.

Subset (flagged by the matcher)	Count	Success@10px	Median error
Unique correlation peak	92	97.8%	0.83 px
Competing (ambiguous) peaks	108	25.9%	73.93 px

This second table is the heart of the submission. The spec-compliant aggregate rate includes an easy population the method nails almost perfectly and an ambiguous population. Because the external fixture labels arbitrary random crops rather than the task’s nearest-centre tie convention, the latter is deliberately an adverse but honest measure. The matcher identifies this risk on its own.

7 Why this is a solid engineering solution

- **Right-sized.** The resolvable subset is solved to roughly one-pixel accuracy by a classical method with two dependencies. We proved, with numbers, that added complexity (a verification stage) does not help — and argued from the data why a learned model cannot either.
- **Honest about limits.** We identify periodic single-view ambiguity as a data-information limit, and separately disclose geometric rotation/distortion as an open, potentially solvable robustness gap.
- **Self-aware.** The ambiguity flag gives a calibrated confidence signal (98% correct when it claims a unique match), which is directly useful in a real inspection workflow.
- **Reproducible.** Fixed seeds, disjoint splits, a results ledger (`results/experiments.csv`), clearly labelled synthetic metrics, and automated tests covering the matcher’s accuracy and its ambiguity detection.

- **Deployable.** Inference needs no internet, no GPU, and no learned weights: it reads a Reference and a Search image and prints the centre coordinates.

The documented executable paths are covered by automated regression tests and public CLI smoke runs.
The result tables point to their seeded synthetic ledgers in the accompanying repository.